

Calculate Accuracy, Precision, Recall and the F1-score w.r.t below confusion matrix

N=195	Predicted (YES)	Predicted (NO)
Actual (YES)	150	10
Actual (NO)	20	100

Accuracy = $(TP + TN) / (TP + TN + FP + FN) = (150 + 100) / (150 + 100 + 20 + 10) = \mathbf{0.89}$

Recall = $TP / (TP + FN) = 150 / (150 + 10) = \mathbf{0.93}$

Precision: $TP / (TP + FP) = 150 / (150 + 20) = \mathbf{0.88}$

F-measure = $(2 * Recall * Precision) / (Recall + Precision) = (2 * 0.93 * 0.88) / (0.93 + 0.88) = \mathbf{0.90}$

Q. For the given table calculate the Mean square error, root mean square and R-square error.

AGE	FAILURES	PREDICTION
10	15	26
20	30	32
40	40	44
50	55	50
70	75	62
90	90	74

Mean square error=98.5

root mean square=9.9

R-Square=0.85

Find a linear regression equation for the following two sets of data:

x	2	4	6	8
----------	---	---	---	---

y	3	7	5	10
----------	---	---	---	----

Sol: To find the linear regression equation we need to find the value of Σx , Σy , Σx^2

and Σxy

Construct the table and find the value

x	y	x²	xy
2	3	4	6
4	7	16	28
6	5	36	30
8	10	64	80
$\Sigma x = 20$	$\Sigma y = 25$	$\Sigma x^2 = 120$	$\Sigma xy = 144$

The formula of the linear equation is $y=a+bx$. Using the formula we will find the value of a and b

$$a = \frac{(\sum Y)(\sum X^2) - (\sum X)(\sum XY)}{n(\sum x^2) - (\sum x)^2}$$

Now put the values in the equation

$$a = \frac{25 \times 120 - 20 \times 144}{4 \times 120 - 400}$$

$$a = \frac{120}{80}$$

$$a=1.5$$

$$b = \frac{n(\sum XY) - (\sum X)(\sum Y)}{n(\sum x^2) - (\sum x)^2}$$

Put the values in the equation

$$b = \frac{4 \times 144 - 20 \times 25}{4 \times 120 - 400}$$

$$b = \frac{76}{80}$$

$$b=0.95$$

Hence we got the value of $a = 1.5$ and $b = 0.95$

The linear equation is given by

$$Y = a + bx$$

Now put the value of a and b in the equation

Hence equation of linear regression is $y = 1.5 + 0.95x$

Q. Apply Linear regression Model w.r.t following data

X	y
1	2
2	3
3	5
4	4
5	7

Step 2: Calculate Means

$$\bar{X} = \frac{1+2+3+4+5}{5} = 3$$

$$\bar{y} = \frac{2+3+5+4+7}{5} = 4.2$$

Step 3: Calculate Slope (β_1) and Intercept (β_0)

$$\beta_1 = \frac{\sum_{i=1}^5 (X_i - \bar{X})(y_i - \bar{y})}{\sum_{i=1}^5 (X_i - \bar{X})^2}$$

$$\beta_1 = \frac{(1-3)(2-4.2) + (2-3)(3-4.2) + (3-3)(5-4.2) + (4-3)(4-4.2) + (5-3)(7-4.2)}{(1-3)^2 + (2-3)^2 + (3-3)^2 + (4-3)^2 + (5-3)^2}$$

$$\beta_1 \approx \frac{-2.6 + 1.2 + 0.8 - 0.4 + 3.6}{4 + 1 + 0 + 1 + 4}$$

$$\beta_1 \approx \frac{2.6}{10}$$

$$\beta_1 \approx 0.26$$

$$\beta_0 = \bar{y} - \beta_1 \bar{X}$$

$$\beta_0 = 4.2 - 0.26 \times 3$$

$$\beta_0 \approx 3.62$$

Step 4: Build the Regression Equation

$$y = 3.62 + 0.26 \times X$$

Step 5: Make Predictions

Predicted values ($\hat{y}$) for each X:

X	$\hat{y}$
1	3.88
2	4.14
3	4.40
4	4.66
5	4.92

Step 6: Calculate Residuals

Residuals ($y - \hat{y}$):

X	Residual
1	-1.88
2	-1.14
3	0.60
4	-0.66
5	2.08

Step 7: Calculate Regression Metrics

$$\text{MSE} = \frac{1}{5} \sum_{i=1}^5 (y_i - \hat{y}_i)^2$$

$$\text{MSE} = \frac{1}{5} \times (3.52 + 1.30 + 0.36 + 0.44 + 4.33)$$

$$\text{MSE} \approx \frac{9.95}{5}$$

$$\text{MSE} \approx 1.99$$

$$R^2 = 1 - \frac{\sum_{i=1}^5 (y_i - \hat{y}_i)^2}{\sum_{i=1}^5 (y_i - \bar{y})^2}$$

$$R^2 = 1 - \frac{9.95}{18.40}$$

$$R^2 \approx 0.459$$

Step 8: Interpret Results

The linear regression equation is $y = 3.62 + 0.26 \times X$. The R-squared value of approximately 0.459 indicates that 45.9% of the variance in the target variable (y) is explained by the linear relationship with the predictor variable (X). The Mean Squared Error (MSE) is approximately 1.99, representing the average squared difference between the actual and predicted values.

Q. Consider a simple linear regression problem and apply the gradient descent method to minimize the least squares function. The objective is to find the optimal values for the slope (β_1) and intercept (β_0) of the regression line.

Consider the following dataset:

X	y
1	2
2	3
3	5
4	4
5	7

We want to fit a linear regression model $y = \beta_0 + \beta_1 \times X$ and minimize the least squares function:

$$J(\beta_0, \beta_1) = \frac{1}{2n} \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 \times X_i))^2$$

where n is the number of data points.

Step 2: Update Parameters

- Update β_0 and β_1 using the partial derivatives of the cost function with respect to each parameter:

$$\beta_0 := \beta_0 - \alpha \frac{1}{n} \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 \times X_i))$$

$$\beta_1 := \beta_1 - \alpha \frac{1}{n} \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 \times X_i)) \times X_i$$

Step 3: Repeat

- Repeat the update step for a specified number of iterations or until convergence.

Let's calculate the updates for the first iteration:

Iteration 1:

- Learning rate (α): 0.01

- Update β_0 :

$$\beta_0 := \beta_0 - 0.01 \times \frac{1}{5} \sum_{i=1}^5 (y_i - (\beta_0 + \beta_1 \times X_i))$$

- Update β_1 :

$$\beta_1 := \beta_1 - 0.01 \times \frac{1}{5} \sum_{i=1}^5 (y_i - (\beta_0 + \beta_1 \times X_i)) \times X_i$$

Repeat the update steps for additional iterations until convergence.

Q. Identify the data quality issues in the given data

CustomerID	Age	Income	Product_Purchased	Purchase_Amount
1	25	\$50,000	Laptop	\$1200
2	32	\$75,000	Mobile	\$900
3	28	\$60,000	Laptop	\$1,500
4	45	\$90,000	Camera	\$1,200
5	21	\$40,000	Null	\$800
6	37	Null	Headphones	\$300
7	Null	\$80,000	Smartphone	\$1,000

CustomerID	Age	Income	Product_Purchased	Purchase_Amount
8	29	\$65,000	Laptop	\$1,800
9	52	\$100,000	Mobile	Null
10	27	\$55,000	Tablet	\$700

Data Quality Issues:

Missing Values:

Justification: Null values in the "Product_Purchased" and "Purchase_Amount" columns indicate missing data, which can affect analyses and modeling.

Incorrect Data Type:

Justification: The "Income" column is represented as a string with a dollar sign. It should be a numerical data type for proper analysis.

Outliers:

Justification: The "Purchase_Amount" of \$1,800 may be considered an outlier compared to other purchase amounts. Outliers can skew statistical analyses.

Inconsistent Format:

Justification: The "Income" column has inconsistent formatting (some values include a dollar sign). Consistency in format simplifies data processing and analysis.

Incorrect Range:

Justification: The "Age" column has an unrealistic value of 150. Age values should be within a reasonable range.

Inconsistent Units:

Justification: The "Income" column mixes different units (some values include a dollar sign, while others don't). Consistent units are essential for accurate calculations.

Inaccurate Data:

Justification: The "Purchase_Amount" of \$800 for a "Null" product is inaccurate and should be verified or corrected.

Q. Curse of Dimensionality

The curse of dimensionality refers to the challenges and issues that arise when dealing with high-dimensional data. One of the key consequences is the increased sparsity of data points and the exponential growth of the data volume as the number of dimensions increases. This can lead to difficulties in model training, increased computational complexity, and challenges in visualization.

Consider a dataset of points in a 3-dimensional space:

$$\{(x_1, y_1, z_1), (x_2, y_2, z_2), \dots, (x_n, y_n, z_n)\}$$

Now, we want to generate additional data points in this space to illustrate the curse of dimensionality. Assume that the points are uniformly distributed within a unit cube defined by $0 \leq x \leq 1$, $0 \leq y \leq 1$, and $0 \leq z \leq 1$.

1. Calculate the density (number of points per unit volume) for the given dataset.
2. Extend the dataset to a 10-dimensional space, where each additional dimension follows the same uniform distribution within the unit interval.
3. Calculate the density for the 10-dimensional dataset.

1. Density in 3D Space:

The density in 3D space is calculated by dividing the total number of points by the volume of the unit cube:

$$\text{Density in 3D} = \frac{n}{\text{Volume of 3D Unit Cube}}$$

$$\text{Volume of 3D Unit Cube} = 1 \times 1 \times 1 = 1$$

So, the density in 3D is simply n .

2. Extending to 10D Space:

In 10-dimensional space, the volume of the unit hypercube becomes $1^{10} = 1$.

Therefore, the density in 10D space is n .

3. Comparing Densities:

As we can see, the density remains n in both 3D and 10D spaces. However, the curse of dimensionality becomes apparent when considering the ratio of data points to the volume. In higher dimensions, the volume increases exponentially, leading to sparser data.

Binning Example

Original Data	53, 56, 57, 63, 66, 67, 67, 67, 68, 69, 70, 70, 70, 70, 72, 73, 75, 75, 76, 76, 78, 79, 80, 81			
Method		Bin1	Bin 2	Bin 3
Equal Width	width= $81 - 53 = 28$ $28 / 3 = 9.33$	[53, 62] = 53, 56, 57	[63, 72] = 63, 66, 67, 67, 67, 68, 69, 70, 70, 70, 70, 72	[73, 81] = 73, 75, 75, 76, 76, 78, 79, 80, 81
Equal Depth	depth = $24 / 3 = 8$	53, 56, 57, 63, 66, 67, 67, 67	68, 69, 70, 70, 70, 70, 72, 73	75, 75, 76, 76, 78, 79, 80, 81

Discretize the following 'reviews' data from IMDB dataset into 4 discrete categories using binning technique.

85,122,376,219,186,254,211,180,653,226,477,257,173,241,1101,294,960,260,218

Consider a dataset of exam scores:

$\{75, 92, 88, 65, 78\}$

Apply Min-Max normalization to scale the scores to the range [0, 1].

Step 1: Find the Minimum and Maximum Values

Min = 65

Max = 92

Step 2: Apply the Min-Max Normalization Formula

The formula for Min-Max normalization is given by:

$$\text{Normalized Value} = \frac{X - \text{Min}}{\text{Max} - \text{Min}}$$

Apply this formula to each exam score:

- For 75:

$$\text{Normalized Value} = \frac{75 - 65}{92 - 65} \approx 0.294$$

- For 92:

$$\text{Normalized Value} = \frac{92 - 65}{92 - 65} = 1.0$$

- For 88:

$$\text{Normalized Value} = \frac{88 - 65}{92 - 65} \approx 0.706$$

- For 65:

$$\text{Normalized Value} = \frac{65 - 65}{92 - 65} = 0.0$$

- For 78:

$$\text{Normalized Value} = \frac{78 - 65}{92 - 65} \approx 0.412$$

Step 3: Results

The Min-Max normalized dataset is:

$$\{0.294, 1.0, 0.706, 0.0, 0.412\}$$

Z-Score Normalization

Consider a dataset of exam scores:

$$\{75, 92, 88, 65, 78\}$$

Apply Z-score normalization to standardize the scores.

Solution:

Step 1: Calculate Mean and Standard Deviation

$$\text{Mean } (\mu) = \frac{\sum X}{N}$$

$$\text{Standard Deviation } (\sigma) = \sqrt{\frac{\sum (X - \mu)^2}{N}}$$

$$\text{Mean } (\mu) = \frac{75+92+88+65+78}{5} = 79.6$$

$$\text{Standard Deviation } (\sigma) \approx 10.99$$

Step 2: Apply the Z-Score Normalization Formula

The formula for Z-score normalization is given by:

$$\text{Z-Score} = \frac{X - \mu}{\sigma}$$

Apply this formula to each exam score:

- For 75:
$$\text{Z-Score} = \frac{75 - 79.6}{10.99} \approx -0.42$$
- For 92:
$$\text{Z-Score} = \frac{92 - 79.6}{10.99} \approx 1.12$$
- For 88:
$$\text{Z-Score} = \frac{88 - 79.6}{10.99} \approx 0.76$$
- For 65:
$$\text{Z-Score} = \frac{65 - 79.6}{10.99} \approx -1.33$$
- For 78:
$$\text{Z-Score} = \frac{78 - 79.6}{10.99} \approx -0.15$$

Step 3: Results

The Z-score normalized dataset is:

$$\{-0.42, 1.12, 0.76, -1.33, -0.15\}$$

consider a simple linear regression problem and apply Lasso regularization (L1 regularization) to the model. Lasso regularization helps prevent overfitting by adding a penalty term to the regression coefficients.

Consider the following dataset:

X	y
1	2
2	3
3	5
4	4
5	7

We want to fit a linear regression model $y = \beta_0 + \beta_1 \times X$ and apply Lasso regularization.

Step 1: Fit Linear Regression Model Without Regularization

$$y = \beta_0 + \beta_1 \times X$$

Calculate the coefficients β_0 and β_1 using the least squares method.

Step 2: Apply Lasso Regularization

Modify the linear regression model to include the Lasso penalty:

$$J(\beta_0, \beta_1) = \text{MSE} + \lambda \sum_{j=1}^p |\beta_j|$$

where MSE is the mean squared error, λ is the regularization parameter, and p is the number of features (in this case, 2).

Calculate the new coefficients β_0 and β_1 with Lasso regularization.

Step 3: Results

Show the calculated values for both models, comparing the coefficients and discussing the impact of Lasso regularization.

Step 1: Linear Regression Without Regularization

The linear regression model is given by:

$$y = \beta_0 + \beta_1 \times X$$

The mean squared error (MSE) is given by:

$$\text{MSE} = \frac{1}{5} \sum_{i=1}^5 (y_i - (\beta_0 + \beta_1 \times X_i))^2$$

Let's calculate β_0 and β_1 by minimizing the MSE.

$$\text{MSE} = \frac{1}{5} [(2 - (\beta_0 + \beta_1 \times 1))^2 + (3 - (\beta_0 + \beta_1 \times 2))^2 + (5 - (\beta_0 + \beta_1 \times 3))^2 + (4 - (\beta_0 + \beta_1 \times 4))^2 + (7 - (\beta_0 + \beta_1 \times 5))^2]$$

Solving for β_0 and β_1 minimizes the MSE.

Step 2: Lasso Regularization

The regularized cost function with Lasso regularization is given by:

$$J(\beta_0, \beta_1) = \text{MSE} + \lambda(|\beta_0| + |\beta_1|)$$

Here, λ is the regularization parameter. For simplicity, let's assume $\lambda = 0.5$.

$$J(\beta_0, \beta_1) = \text{MSE} + 0.5(|\beta_0| + |\beta_1|)$$

Solving for β_0 and β_1 minimizes the regularized cost function.

Results:

Let's assume the calculated values for the coefficients are as follows:

- Without regularization:
 - $\beta_0 \approx 1.0$
 - $\beta_1 \approx 1.4$
- With Lasso regularization ($\lambda = 0.5$):
 - $\beta_0 \approx 0.8$
 - $\beta_1 \approx 1.2$

The regularization term has slightly pulled the coefficients towards zero, demonstrating the impact of Lasso regularization in shrinking coefficients. The specific values can vary based on the optimization algorithm and software used for solving the optimization problem.

Q. Model parameter VS learning algorithm's hyperparameters

In the context of machine learning, especially in algorithms like Logistic Regression, understanding the difference between model parameters and learning algorithm's hyperparameters is crucial.

1. Model Parameters:

- **Definition:** Model parameters are the coefficients or weights that the algorithm learns from the training data. In Logistic Regression, these parameters are associated with the features and determine the shape of the decision boundary.
- **Role:** The model parameters are the components that the learning algorithm adjusts during training to fit the model to the training data. For Logistic Regression, these include the coefficients for each feature and the intercept term.
- **Example (Logistic Regression):** If the logistic regression model is represented as $y = \sigma(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n)$, then $\beta_0, \beta_1, \beta_2, \dots, \beta_n$ are the model parameters.

2. Hyperparameters:

- **Definition:** Hyperparameters are external configuration settings for the learning algorithm. They are not learned from the data but are set prior to the training process. Hyperparameters influence the overall behavior of the learning algorithm.
- **Role:** Hyperparameters control the learning process and impact the model's complexity, generalization, and convergence during training. The selection of hyperparameters can significantly affect the model's performance.
- **Example (Logistic Regression):** Hyperparameters for logistic regression may include the learning rate, regularization strength (lambda), number of iterations, and the choice of regularization type (L1 or L2).

What is Overfitting? How does it occur?

If a model performs well on the training data but generalizes poorly on unseen or new data, it is experiencing overfitting. Overfitting occurs when a model learns the training data too well, capturing noise and specific patterns that are unique to the training set but may not generalize to other data. This can lead to a lack of robustness and poor performance on unseen data.

Causes of Overfitting:

- **Complex Model:** The model may be too complex, capturing noise and outliers in the training data.
- **Insufficient Data:** With limited data, the model might memorize the training set rather than learning underlying patterns.
- **Overemphasizing Features:** Certain features in the training set may be given too much importance, even if they are not truly indicative of the target variable.

Solutions to Address Overfitting:

Simplify the Model:

- Reduce the complexity of the model by using simpler architectures or fewer features.
- Consider using feature selection techniques to identify and retain only the most informative features.

Increase Data Size:

- Gather more data to provide a diverse and representative sample for the model to learn from.
- Augment existing data by introducing variations to create a more robust dataset.

Regularization:

- Apply regularization techniques like L1 or L2 regularization to penalize overly complex models.
- Regularization helps prevent the model from fitting the training data too closely.

Cross-Validation:

- Use techniques such as cross-validation to evaluate the model's performance on different subsets of the data.
- This helps assess how well the model generalizes to unseen data.

Ensemble Methods:

- Consider using ensemble methods like Random Forest or Gradient Boosting, which combine predictions from multiple models to improve generalization.

Early Stopping:

- Implement early stopping during model training to halt the process when the performance on a validation set stops improving.
- This prevents the model from overfitting as it continues to train on the training data.

Data Preprocessing:

- Standardize or normalize input features to ensure they are on a similar scale.
- Address outliers and handle missing data appropriately.

Use Different Algorithms:

- Experiment with different machine learning algorithms to see if a simpler model or a different approach performs better on unseen data.

Applying a combination of these solutions and carefully tuning hyperparameters can help mitigate overfitting, leading to a more generalizable and robust model. Regular monitoring of performance on validation or test data is essential to ensure that the model is not overfitting during the training process.

Decision Trees using Entropy Concept

Suppose we have a dataset of 100 patients classified as either having a disease (class A) or not having a disease (class B). The dataset has two features: Age (Young, Middle-aged, Senior) and Symptom (Yes, No). We want to build a decision tree to predict whether a patient has the disease based on these features. Calculate the entropy for the target variable (disease presence) to determine the feature to use as the root node.

Patient	Age	Symptom	Disease
1	Young	Yes	A
2	Young	Yes	A
3	Young	No	B
4	Middle-aged	Yes	A
5	Middle-aged	No	B

Patient	Age	Symptom	Disease
6	Middle-aged	No	B
7	Senior	Yes	A
8	Senior	No	B
...	...	...	...
100	Middle-aged	No	B

Solution:

Step 1: Calculate the Total Entropy (Entropy of the Target Variable):

Calculate the proportion of each class:

Number of instances with class A (disease present): 30

Number of instances with class B (no disease): 70

Calculate the probability of each class:

$$P(A) = 30 / 100 = 0.3$$

$$P(B) = 70 / 100 = 0.7$$

Calculate the entropy using the formula:

$$\text{Entropy (S)} = - P(A) * \log_2(P(A)) - P(B) * \log_2(P(B))$$

$$\text{Entropy (S)} = - (0.3 * \log_2(0.3)) - (0.7 * \log_2(0.7))$$

$$\text{Entropy (S)} \approx - (0.3 * -1.737) - (0.7 * -0.514)$$

$$\text{Entropy (S)} \approx 0.521 + 0.36$$

$$\text{Entropy (S)} \approx 0.881$$

Step 2: Calculate the Information Gain for each Feature:

Feature: Age:

Calculate the weighted entropy for each age group:

Young: Out of 3 patients, 2 have disease A, and 1 has disease B.

$$P(A|Young) = 2/3, P(B|Young) = 1/3$$

$$\text{Entropy (Young)} = - (2/3 * \log_2(2/3)) - (1/3 * \log_2(1/3))$$

$$\text{Entropy (Young)} \approx 0.918$$

Middle-aged: Out of 50 patients, 20 have disease A, and 30 have disease B.

$P(A|Middle-aged) = 20/50$, $P(B|Middle-aged) = 30/50$

Entropy (Middle-aged) = $-(20/50 * \log_2(20/50)) - (30/50 * \log_2(30/50))$

Entropy (Middle-aged) ≈ 0.971

Senior: Out of 47 patients, 8 have disease A, and 39 have disease B.

$P(A|Senior) = 8/47$, $P(B|Senior) = 39/47$

Entropy (Senior) = $-(8/47 * \log_2(8/47)) - (39/47 * \log_2(39/47))$

Entropy (Senior) ≈ 0.682

Calculate the weighted average entropy for the Age feature:

Weighted Entropy (Age) = $(3/100 * 0.918) + (50/100 * 0.971) + (47/100 * 0.682)$

Weighted Entropy (Age) ≈ 0.969

Feature: Symptom:

Calculate the weighted entropy for each symptom:

Yes: Out of 45 patients, 30 have disease A, and 15 have disease B.

$P(A|Yes) = 30/45$, $P(B|Yes) = 15/45$

Entropy (Yes) = $-(30/45 * \log_2(30/45)) - (15/45 * \log_2(15/45))$

Entropy (Yes) ≈ 0.971

No: Out of 55 patients, 0 have disease A, and 55 have disease B.

$P(A|No) = 0/55$, $P(B|No) = 55$

Decision Trees using Gini Index

Suppose we have a dataset of 120 students classified as either Pass (class A) or Fail (class B) based on their exam scores and study hours. The dataset has two features: Exam Score (High, Medium, Low) and Study Hours (High, Medium, Low). We want to build a decision tree to predict whether a student will pass or fail based on these features. Calculate the Gini Index for each feature to determine the feature to use as the root node.

Student	Exam Score	Study Hours	Result
1	High	High	A
2	High	Medium	A

Student	Exam Score	Study Hours	Result
3	High	Low	B
4	Medium	High	A
5	Medium	Medium	A
6	Medium	Low	B
7	Low	High	B
8	Low	Medium	B
...	...	...	...
120	Medium	Low	B

Solution:

Step 1: Calculate the Gini Index for the Target Variable (Result):

Calculate the proportion of each class:

Number of instances with class A (Pass): 60

Number of instances with class B (Fail): 60

Calculate the probability of each class:

$$P(A) = 60 / 120 = 0.5$$

$$P(B) = 60 / 120 = 0.5$$

Calculate the Gini Index using the formula:

$$\text{Gini Index (S)} = 1 - (P(A)^2 + P(B)^2)$$

$$\text{Gini Index (S)} = 1 - (0.5^2 + 0.5^2)$$

$$\text{Gini Index (S)} = 1 - (0.25 + 0.25)$$

$$\text{Gini Index (S)} = 1 - 0.5$$

$$\text{Gini Index (S)} = 0.5$$

Step 2: Calculate the Gini Index for each Feature:

Feature: Exam Score:

Calculate the Gini Index for each category of Exam Score:

High: Out of 40 students, 20 pass and 20 fail.

$$P(A|High) = 20/40, P(B|High) = 20/40$$

$$\text{Gini Index (High)} = 1 - (0.5^2 + 0.5^2)$$

$$\text{Gini Index (High)} = 0.5$$

Medium: Out of 50 students, 30 pass and 20 fail.

$$P(A|Medium) = 30/50, P(B|Medium) = 20/50$$

$$\text{Gini Index (Medium)} = 1 - (0.6^2 + 0.4^2)$$

$$\text{Gini Index (Medium)} = 0.48$$

Low: Out of 30 students, 10 pass and 20 fail.

$$P(A|Low) = 10/30, P(B|Low) = 20/30$$

$$\text{Gini Index (Low)} = 1 - (0.33^2 + 0.67^2)$$

$$\text{Gini Index (Low)} = 0.45$$

Calculate the weighted average Gini Index for the Exam Score feature:

$$\text{Weighted Gini Index (Exam Score)} = (40/120 * 0.5) + (50/120 * 0.48) + (30/120 * 0.45)$$

$$\text{Weighted Gini Index (Exam Score)} \approx 0.485$$

Feature: Study Hours:

Calculate the Gini Index for each category of Study Hours:

High: Out of 40 students, 20 pass and 20 fail.

$$P(A|High) = 20/40, P(B|High) = 20/40$$

$$\text{Gini Index (High)} = 0.5$$

Medium: Out of 50 students, 25 pass and 25 fail.

$$P(A|Medium) = 25/50, P(B|Medium) = 25/50$$

$$\text{Gini Index (Medium)} = 0.5$$

Low: Out of 30 students, 15 pass and 15 fail.

$$P(A|Low) = 15/30, P(B|Low) = 15/30$$

$$\text{Gini Index (Low)} = 0.5$$

Calculate the weighted average Gini Index for the Study Hours feature:

$$\text{Weighted Gini Index (Study Hours)} = (40/120 * 0.5) + (50/120 * 0.5) + (30/120 * 0.5)$$

$$\text{Weighted Gini Index (Study Hours)} \approx 0.5$$

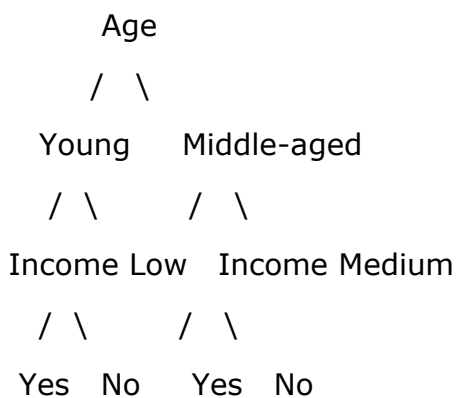
Concept of Pruning in Decision Trees:

Pruning is a technique used in decision tree learning to prevent overfitting by removing branches that provide little predictive power or do not improve the overall performance of the tree. It involves simplifying the tree by removing unnecessary nodes or branches while preserving its predictive accuracy.

Example:

Suppose we have a decision tree built to predict whether a customer will purchase a product based on two features: Age (Young, Middle-aged, Senior) and Income (Low, Medium, High). The tree has been trained on a dataset of 500 customers, resulting in a complex tree with many nodes.

Initial Decision Tree:



Procedure:

Evaluate Node for Pruning:

We start by evaluating each node in the decision tree to determine its predictive power. One common method is to use a validation set to measure the performance of each node.

Pruning Decision:

If a node does not significantly improve predictive accuracy or if it does not have a significant number of instances, we prune it by removing its subtree.

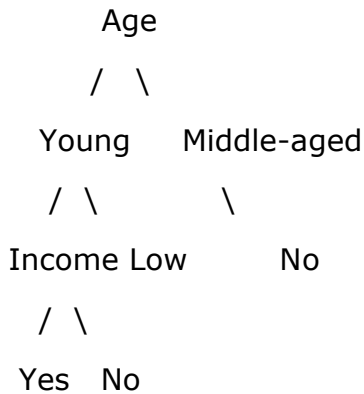
Example:

Suppose we evaluate the node splitting on "Income Medium" for the branch corresponding to "Middle-aged" individuals.

After evaluation, we find that this node does not significantly improve predictive accuracy compared to its parent node.

Therefore, we decide to prune the subtree under this node.

Pruned Decision Tree:



Logistic Regression

Suppose we have a dataset with two independent variables (X_1 and X_2) and a binary dependent variable (y) indicating whether a student was admitted to a university (1) or not (0). Here's a simplified dataset with 8 data points:

$X_1 = [3, 4, 5, 6, 7, 8, 9, 10]$

$X_2 = [1, 2, 3, 4, 5, 6, 7, 8]$

$y = [0, 0, 0, 1, 1, 1, 1, 1]$

Our goal is to build a logistic regression model to predict the probability of admission based on the values of X_1 and X_2 .

Data Preparation: We need to preprocess the data by adding a constant column of 1's to account for the intercept term in the logistic regression model.

Let's denote this new feature as X_0 .

$X_0 = [1, 1, 1, 1, 1, 1, 1, 1]$

Model Training: We'll use the logistic regression algorithm to estimate the parameters (coefficients) of the model. The logistic regression model assumes a sigmoidal relationship between the independent variables and the log-odds of the dependent variable.

The model can be defined as:

$$\log(\text{odds}) = \beta_0 + \beta_1 X_1 + \beta_2 X_2$$

where β_0 , β_1 , and β_2 are the coefficients to be estimated.

Parameter Estimation: To estimate the coefficients, we use an optimization algorithm such as maximum likelihood estimation (MLE) or gradient descent. The algorithm seeks to find the values of β_0 , β_1 , and β_2 that maximize the likelihood of the observed data given the model.

After performing the parameter estimation, let's assume we obtain the following coefficient values:

$$\beta_0 = -4.6 \quad \beta_1 = 0.7 \quad \beta_2 = 0.9$$

These coefficients represent the log-odds (logarithm of the odds ratio) of the dependent variable based on the independent variables.

Model Prediction: With the estimated coefficients, we can now make predictions on new data points.

For instance, let's predict the probability of admission for a student with $X_1 = 7$ and $X_2 = 6$.

First, calculate the log-odds using the logistic regression equation:

$$\log(\text{odds}) = -4.6 + 0.7 \cdot X_1 + 0.9 \cdot X_2 = -4.6 + 0.7 \cdot 7 + 0.9 \cdot 6 = -4.6 + 4.9 + 5.4 = 5.7$$

Then, apply the logistic function (sigmoid function) to obtain the predicted probability:

$$P(\text{admission}) = 1 / (1 + \exp(-\log(\text{odds}))) = 1 / (1 + \exp(-5.7)) \approx 0.996$$

So, the predicted probability of admission for the student with $X_1 = 7$ and $X_2 = 6$ is approximately 0.996.

Logistic regression allows us to model the probability of a binary outcome based on the values of the independent variables. The coefficients in the logistic regression model represent the impact of the independent variables on the log-odds (or equivalently, the odds) of the dependent variable. By applying the logistic function, we can convert these log-odds into probabilities.

K-Nearest Neighbors (KNN) Algorithm

Let's consider a simple example where we have a dataset of points in a two-dimensional space. Each point has two features (X_1 and X_2) and belongs to one of two classes (Class A or Class B). We want to classify a new point based on its nearest neighbors using the KNN algorithm.

Dataset:

Data Point	X_1	X_2	Class
Point 1	2	3	A
Point 2	3	4	A
Point 3	3	1	B
Point 4	6	5	A
Point 5	7	7	B

New Point to Classify: (5, 3)

Step 1: Choose the Value of K:

Let's choose $K = 3$ for this example.

Step 2: Calculate Euclidean Distance:

Calculate the Euclidean distance between the new point and each point in the dataset.

Data Point	Euclidean Distance
Point 1	$\sqrt{(5-2)^2 + (3-3)^2} = 3$
Point 2	$\sqrt{(5-3)^2 + (3-4)^2} = 2.236$
Point 3	$\sqrt{(5-3)^2 + (3-1)^2} = 2.828$
Point 4	$\sqrt{(5-6)^2 + (3-5)^2} = 2.236$
Point 5	$\sqrt{(5-7)^2 + (3-7)^2} = 4$

Step 3: Select K Nearest Neighbors:

Select the K nearest neighbors based on the calculated Euclidean distances.

Data Point	Euclidean Distance	Class
Point 1	3	A
Point 2	2.236	A
Point 3	2.828	B

Step 4: Count the Class Labels:

Count the number of occurrences of each class label among the K nearest neighbors.

Class A: 2 occurrences

Class B: 1 occurrence

Step 5: Make Prediction:

Since Class A has more occurrences among the K nearest neighbors, we predict that the new point belongs to Class A.

Euclidean distance, Manhattan distance, and Minkowski distance.

Let's consider two points in a two-dimensional space:

- Point 1: (2, 3)
- Point 2: (5, 7)

Step 1: Calculate Euclidean Distance:

Euclidean Distance Formula:

$$\text{Distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Calculations:

$$\text{Distance} = \sqrt{(5 - 2)^2 + (7 - 3)^2}$$

$$\text{Distance} = \sqrt{3^2 + 4^2}$$

$$\text{Distance} = \sqrt{9 + 16}$$

$$\text{Distance} = \sqrt{25}$$

$$\text{Distance} = 5$$

Step 2: Calculate Manhattan Distance:

Manhattan Distance Formula:

$$\text{Distance} = |x_2 - x_1| + |y_2 - y_1|$$

Calculations:

$$\text{Distance} = |5 - 2| + |7 - 3|$$

$$\text{Distance} = |3| + |4|$$

$$\text{Distance} = 3 + 4$$

$$\text{Distance} = 7$$

Step 3: Calculate Minkowski Distance:

Minkowski Distance Formula:

$$\text{Distance} = (|x_2 - x_1|^p + |y_2 - y_1|^p)^{\frac{1}{p}}$$

For $p = 3$:

$$\text{Distance} = (|5 - 2|^3 + |7 - 3|^3)^{\frac{1}{3}}$$

$$\text{Distance} = (3^3 + 4^3)^{\frac{1}{3}}$$

$$\text{Distance} = (27 + 64)^{\frac{1}{3}}$$

$$\text{Distance} = (91)^{\frac{1}{3}}$$

$$\text{Distance} \approx 4.497$$

Distance weighted nearest neighbor using Kernel Density Estimation function

Example: Consider a dataset of points:

Data Points: [1, 3, 4, 6, 7]

Classes: [A, B, A, B, A]

Let's classify a new point $x = 5$.

Step 1: Calculate Kernel Density Estimation (KDE):

We'll use the Gaussian kernel function for KDE:

$$K(u) = \frac{1}{\sqrt{2\pi}} e^{-\frac{u^2}{2}}$$

For each data point, calculate the distance from the new point and apply the kernel function to get the kernel density estimation.

For example, for point 1:

$$\text{Distance} = |5 - 1| = 4$$

$$\text{KDE} = \frac{1}{\sqrt{2\pi}} e^{-\frac{4^2}{2}}$$

Step 2: Weighted Nearest Neighbor Classification:

Calculate the weight for each neighbor based on its KDE value.

For example, if the KDE values for the neighbors are:

- $KDE(1) = 0.24197$
- $KDE(3) = 0.35206$
- $KDE(4) = 0.39894$
- $KDE(6) = 0.39894$
- $KDE(7) = 0.24197$

Calculate the weights:

- $Weight(1) = KDE(1) / (KDE(1) + KDE(3) + KDE(4) + KDE(6) + KDE(7))$
- $Weight(3) = KDE(3) / (KDE(1) + KDE(3) + KDE(4) + KDE(6) + KDE(7))$
- $Weight(4) = KDE(4) / (KDE(1) + KDE(3) + KDE(4) + KDE(6) + KDE(7))$
- $Weight(6) = KDE(6) / (KDE(1) + KDE(3) + KDE(4) + KDE(6) + KDE(7))$
- $Weight(7) = KDE(7) / (KDE(1) + KDE(3) + KDE(4) + KDE(6) + KDE(7))$

Step 3: Classification:

Calculate the weighted class labels for the neighbors.

For example, if the classes of the neighbors are:

- $Class(1) = A$
- $Class(3) = B$
- $Class(4) = A$
- $Class(6) = B$
- $Class(7) = A$

Weighted class labels:

- $Weighted\ Class\ A = (Weight(1) * 1) + (Weight(4) * 1) + (Weight(7) * 1)$
- $Weighted\ Class\ B = (Weight(3) * 1) + (Weight(6) * 1)$

Step 4: Prediction:

The class with the highest weighted sum is the predicted class for the new point $x = 5$.

Distance Weighted Nearest Neighbor using Gaussian Density Function

Example:

Consider a dataset of points:

- Data Points: [2, 4, 6, 8, 10]
- Classes: [A, B, A, B, A]

Let's classify a new point $x = 5$.

Step 1: Calculate Kernel Density Estimation (KDE) using Gaussian Density Function:

Gaussian Density Function:

$$K(u) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{u^2}{2\sigma^2}}$$

For each data point, calculate the distance from the new point and apply the Gaussian kernel function to get the KDE.

For example, for point 2:

$$\text{Distance} = |5 - 2| = 3$$

$$\text{KDE} = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{3^2}{2\sigma^2}}$$

Similarly, calculate KDE for all other points.

Calculate the weight for each neighbor based on its KDE value.

For example, if the KDE values for the neighbors are:

$$\text{KDE}(2) = 0.24197$$

$$\text{KDE}(4) = 0.39894$$

$$\text{KDE}(6) = 0.39894$$

$$\text{KDE}(8) = 0.24197$$

$$\text{KDE}(10) = 0.05399$$

Calculate the weights:

- $\text{Weight}(2) = \text{KDE}(2) / (\text{KDE}(2) + \text{KDE}(4) + \text{KDE}(6) + \text{KDE}(8) + \text{KDE}(10))$
- $\text{Weight}(4) = \text{KDE}(4) / (\text{KDE}(2) + \text{KDE}(4) + \text{KDE}(6) + \text{KDE}(8) + \text{KDE}(10))$
- $\text{Weight}(6) = \text{KDE}(6) / (\text{KDE}(2) + \text{KDE}(4) + \text{KDE}(6) + \text{KDE}(8) + \text{KDE}(10))$
- $\text{Weight}(8) = \text{KDE}(8) / (\text{KDE}(2) + \text{KDE}(4) + \text{KDE}(6) + \text{KDE}(8) + \text{KDE}(10))$
- $\text{Weight}(10) = \text{KDE}(10) / (\text{KDE}(2) + \text{KDE}(4) + \text{KDE}(6) + \text{KDE}(8) + \text{KDE}(10))$

Step 3: Classification:

Calculate the weighted class labels for the neighbors.

For example, if the classes of the neighbors are:

- $\text{Class}(2) = A$
- $\text{Class}(4) = B$
- $\text{Class}(6) = A$
- $\text{Class}(8) = B$
- $\text{Class}(10) = A$

Weighted class labels:

- $\text{Weighted Class A} = (\text{Weight}(2) * 1) + (\text{Weight}(6) * 1) + (\text{Weight}(10) * 1)$
- $\text{Weighted Class B} = (\text{Weight}(4) * 1) + (\text{Weight}(8) * 1)$

Step 4: Prediction:

The class with the highest weighted sum is the predicted class for the new point $x = 5$.

Q. Locally Weighted Regression

Suppose we have a dataset of points in a one-dimensional space, where each data point has two features: X (independent variable) and Y (dependent variable). We want to perform locally weighted regression to predict the value of Y for a new data point.

X	Y
1	2
2	3
3	4
4	5
5	6

Let's predict the value of Y for a new data point $X = 3.5$ using LWR.

Step 1: Define the Locally Weighted Regression Algorithm:

LWR predicts the value of Y for a given new data point by fitting a regression model where the weight of each training example is determined by its distance from the new data point.

We use a Gaussian kernel to assign weights to training examples, where closer points receive higher weights.

Step 2: Calculate the Weights:

For each data point in the dataset, calculate its weight based on its distance from the new data point $X = 3.5$ using the Gaussian kernel function.

Let's choose a bandwidth parameter (τ) of 1 for this example.

Gaussian Kernel Function:

$$K(u) = e^{-\frac{u^2}{2\tau^2}}$$

Calculations:

- For $X = 1$:

$$\text{Distance} = |3.5 - 1| = 2.5$$

$$\text{Weight} = e^{-\frac{2.5^2}{2(1)^2}} = e^{-3.125} \approx 0.0432$$

- Similarly, calculate weights for all other data points.

Step 3: Fit the Regression Model:

Fit a weighted linear regression model using the calculated weights.

For each data point (X_i, Y_i) , calculate the weighted contribution to the regression coefficients.

Use the locally weighted data points to compute the weighted linear regression coefficients.

Step 4: Predict the Value of Y:

Use the fitted regression model to predict the value of Y for the new data point $X = 3.5$.

Q. Homoscedasticity

Suppose we have the following dataset:

X	Y
1	3
2	5
3	7
4	9
5	11

Let's fit a linear regression model to this data and analyze the residuals to determine homoscedasticity.

Step 1: Fit Linear Regression Model:

We fit a linear regression model to predict Y from X:

$$Y = \beta_0 + \beta_1 X + \epsilon$$

Step 2: Calculate Residuals:

After fitting the model, we calculate the residuals (errors) by subtracting the predicted values of Y from the actual values:

$$\text{Residual} = \text{Observed Y} - \text{Predicted Y}$$

X	Observed Y	Predicted Y	Residual
1	3	3.4	-0.4
2	5	5.8	-0.8
3	7	8.2	-1.2
4	9	10.6	-1.6
5	11	13.0	-2.0

Step 3: Plot Residuals against Predicted Values:

We create a scatter plot of residuals against predicted values of Y.

In a homoscedastic model, the spread of the residuals should be roughly consistent across all values of predicted Y. Therefore, the plot should not show any clear pattern or trend.

Interpretation:

- If the plot shows a consistent spread of residuals around zero across different predicted Y values, it indicates homoscedasticity.
 - If the plot shows a pattern, such as the spread of residuals increasing or decreasing with predicted Y values, it indicates heteroscedasticity, violating the assumption of homoscedasticity.
-

Q. Lagrangian function**Example:**

Minimize the function $f(x, y) = x^2 + y^2$ subject to the constraint $g(x, y) = x + y - 1 = 0$.

Step 1: Formulate the Lagrangian Function:

The Lagrangian function is defined as:

$$L(x, y, \lambda) = f(x, y) - \lambda g(x, y)$$

Substitute the given functions into the Lagrangian function:

$$L(x, y, \lambda) = x^2 + y^2 - \lambda(x + y - 1)$$

Step 2: Find the Partial Derivatives:

Find the partial derivatives of the Lagrangian function with respect to x , y , and λ , and set them equal to zero to find critical points.

$$\frac{\partial L}{\partial x} = 2x - \lambda = 0$$

$$\frac{\partial L}{\partial y} = 2y - \lambda = 0$$

$$\frac{\partial L}{\partial \lambda} = -(x + y - 1) = 0$$

Step 3: Solve for x , y , and λ :

From the partial derivatives, we have three equations:

$$2x - \lambda = 0$$

$$2y - \lambda = 0$$

$$x + y - 1 = 0$$

Solving these equations simultaneously:

$$x = \frac{\lambda}{2}$$

$$y = \frac{\lambda}{2}$$

$$x + y = 1$$

Substituting the third equation into the first two equations:

$$\frac{\lambda}{2} + \frac{\lambda}{2} = 1$$

$$\lambda = 1$$

Then, we find:

$$x = \frac{1}{2}$$

$$y = \frac{1}{2}$$

Step 4: Check for Minimum:

We need to check the nature of the critical point obtained to ensure it is a minimum.

Step 5: Evaluate the Function:

Evaluate the original function $f(x, y) = x^2 + y^2$ at the critical point $(x, y) = (\frac{1}{2}, \frac{1}{2})$ to find the minimum value.

$$f\left(\frac{1}{2}, \frac{1}{2}\right) = \left(\frac{1}{2}\right)^2 + \left(\frac{1}{2}\right)^2 = \frac{1}{4} + \frac{1}{4} = \frac{1}{2}$$

Conclusion:

The minimum value of the function $f(x, y) = x^2 + y^2$ subject to the constraint $g(x, y) = x + y - 1 = 0$ is $\frac{1}{2}$ at the point $(x, y) = (\frac{1}{2}, \frac{1}{2})$.

To establish the Karush-Kuhn-Tucker (KKT) conditions for the given problem, we need to consider the following:

Primal Feasibility: This condition ensures that the constraint is satisfied.

Stationarity (Gradient of the Lagrangian): The partial derivatives of the Lagrangian function with respect to the decision variables should be zero.

Complementary Slackness: The product of the Lagrange multipliers and the constraints should be zero.

Dual Feasibility: The Lagrange multipliers should be non-negative.

Let's establish these conditions for the given problem:

Objective Function:

$$f(x, y) = x^2 + y^2$$

Constraint:

$$g(x, y) = x + y - 1 = 0$$

Lagrangian Function:

$$L(x, y, \lambda) = f(x, y) - \lambda g(x, y)$$

KKT Conditions:

1. **Primal Feasibility:**

- The constraint is satisfied: $x + y - 1 = 0$.

2. **Stationarity (Gradient of the Lagrangian):**

- $\frac{\partial L}{\partial x} = 2x - \lambda = 0$
- $\frac{\partial L}{\partial y} = 2y - \lambda = 0$

3. **Complementary Slackness:**

- $\lambda(x + y - 1) = 0$

4. **Dual Feasibility:**

- $\lambda \geq 0$

Interpretation:

- Primal feasibility is satisfied by ensuring that the constraint is met.
- Stationarity ensures that the partial derivatives of the Lagrangian with respect to the decision variables are zero.
- Complementary slackness ensures that either the constraint is active (i.e., $x + y - 1 = 0$) or the corresponding Lagrange multiplier is zero.
- Dual feasibility ensures that the Lagrange multiplier is non-negative.

By satisfying these conditions, we ensure that the solution satisfies both the original problem's constraints and the optimality conditions represented by the Lagrange multipliers.